

E2E Automation Framework

Complete Implementation Guide

Java + Selenium + TestNG + Maven + ExtentReports

IDE	Eclipse (Maven Project)
Target Site	saucedemo.com (public e-commerce demo)
Design Patterns	POM + PageFactory + Singleton + Factory + Fluent
Test Framework	TestNG with DataProviders & Parallel Execution
Reporting	ExtentReports + Log4j2 / SLF4J
Data-Driven	Excel / JSON / CSV via Apache POI
Cross-Browser	Chrome, Firefox, Edge
CI/CD	Jenkins + GitHub Actions (Jenkinsfile)
Bonus Features	Retry Mechanism, Screenshots on Failure, Env Config

This document covers every phase from project creation to CI/CD pipeline.

Table of Contents

Phase 1	Eclipse Project Setup & Maven Configuration
Phase 2	pom.xml – All Dependencies
Phase 3	Project Folder Structure
Phase 4	config.properties & ConfigReader
Phase 5	BrowserFactory – Cross-Browser (Chrome/Firefox/Edge)
Phase 6	BaseDriver – Singleton WebDriver
Phase 7	BasePage – Fluent + Reusable Actions
Phase 8	Page Objects (POM + PageFactory)
8a	LoginPage.java
8b	HomePage.java
8c	SearchResultsPage.java
8d	ProductPage.java
8e	CartPage.java
8f	CheckoutPage.java
8g	OrderConfirmationPage.java
Phase 9	Test Data – Excel & JSON
Phase 10	Test Classes
10a	LoginTest.java
10b	SearchTest.java
10c	CartTest.java
10d	OrderTest.java
Phase 11	TestListener – ExtentReports + Screenshots
Phase 12	testng.xml – Suite Configuration
Phase 13	log4j2.xml – Logging Configuration
Phase 14	ScreenshotUtils & RetryAnalyzer
Phase 15	ExcelUtils – Data-Driven Testing
Phase 16	Jenkinsfile – CI/CD Pipeline
Phase 17	GitHub Actions Workflow
Phase 18	Run Instructions & Troubleshooting

Step-by-Step Eclipse Setup

Open Eclipse: Launch Eclipse IDE. If not installed, download Eclipse IDE for Java Developers from eclipse.org

Create Maven Project: File → New → Project → Maven → Maven Project → Next

Skip Archetype: Check 'Create a simple project (skip archetype selection)' → Next

Set Coordinates: Group Id: com.ecommerce | Artifact Id: EcommerceAutomation | Version: 1.0.0 | Packaging: jar → Finish

Set Java Version: Right-click project → Properties → Java Build Path → Libraries → JRE System Library → set to JavaSE-11 or 17

Enable Maven Nature: If not auto-added: Right-click → Configure → Convert to Maven Project

Expected Project Layout After Creation

```
EcommerceAutomation/  
  
  ■■■ src/  
  
    ■ ■■■ main/  
  
      ■ ■ ■■■ java/  
  
        ■ ■■■ test/  
  
          ■ ■■■ java/  
  
        ■■■ pom.xml  
  
        ■■■ target/ (auto-generated by Maven)
```

Create Source Folders Manually in Eclipse

Right-click on src/main/java → New → Package. Create these packages:

- com.ecommerce.base
- com.ecommerce.pages
- com.ecommerce.factory
- com.ecommerce.utils

Right-click on src/test/java → New → Package. Create:

- com.ecommerce.tests
- com.ecommerce.listeners

Right-click src/test → New → Folder → Name it 'resources'. Inside create:

- config.properties

- testdata.json
- log4j2.xml

All Maven dependencies and plugins required

FILE: pom.xml (root of project)

```
<?xml version="1.0" encoding="UTF-8"?>

<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>com.ecommerce</groupId>

  <artifactId>EcommerceAutomation</artifactId>

  <version>1.0.0</version>

  <packaging>jar</packaging>

  <properties>

    <maven.compiler.source>11</maven.compiler.source>

    <maven.compiler.target>11</maven.compiler.target>

    <selenium.version>4.18.1</selenium.version>

    <testng.version>7.9.0</testng.version>

    <extentreports.version>5.1.1</extentreports.version>

    <poi.version>5.2.5</poi.version>

    <log4j.version>2.22.1</log4j.version>

  </properties>

  <dependencies>

    <!-- Selenium WebDriver -->

    <dependency>

      <groupId>org.seleniumhq.selenium</groupId>
```

```

<artifactId>selenium-java</artifactId>

<version>${selenium.version}</version>

</dependency>

<!-- WebDriverManager (auto-downloads chromedriver etc.) -->

<dependency>

<groupId>io.github.bonigarcia</groupId>

<artifactId>webdrivermanager</artifactId>

<version>5.7.0</version>

</dependency>

<!-- TestNG -->

<dependency>

<groupId>org.testng</groupId>

<artifactId>testng</artifactId>

<version>${testng.version}</version>

<scope>test</scope>

</dependency>

<!-- ExtentReports -->

<dependency>

<groupId>com.aventstack</groupId>

<artifactId>extentreports</artifactId>

<version>${extentreports.version}</version>

</dependency>

<!-- Apache POI for Excel -->

<dependency>

<groupId>org.apache.poi</groupId>

<artifactId>poi-ooxml</artifactId>

<version>${poi.version}</version>

</dependency>

```

```
<!-- Log4j2 -->

<dependency>

<groupId>org.apache.logging.log4j</groupId>

<artifactId>log4j-core</artifactId>

<version>${log4j.version}</version>

</dependency>

<dependency>

<groupId>org.apache.logging.log4j</groupId>

<artifactId>log4j-slf4j2-impl</artifactId>

<version>${log4j.version}</version>

</dependency>

<!-- JSON (Gson) -->

<dependency>

<groupId>com.google.code.gson</groupId>

<artifactId>gson</artifactId>

<version>2.10.1</version>

</dependency>

</dependencies>

<build>

<plugins>

<!-- Maven Surefire - runs TestNG -->

<plugin>

<groupId>org.apache.maven.plugins</groupId>

<artifactId>maven-surefire-plugin</artifactId>

<version>3.2.5</version>

<configuration>

<suiteXmlFiles>
```

```
<suiteXmlFile>testng.xml</suiteXmlFile>

</suiteXmlFiles>

</configuration>

</plugin>

<!-- Maven Compiler -->

<plugin>

<groupId>org.apache.maven.plugins</groupId>

<artifactId>maven-compiler-plugin</artifactId>

<version>3.12.1</version>

</plugin>

</plugins>

</build>

</project>
```

NOTE: After pasting pom.xml, right-click the project in Eclipse → Maven → Update Project (Alt+F5) to download all dependencies.

EcommerceAutomation/

■ ■ ■ pom.xml

■ ■ ■ testng.xml

■ ■ ■ Jenkinsfile

■ ■ ■ .github/

■ ■ ■ workflows/

■ ■ ■ ci.yml

■ ■ ■ src/

■ ■ ■ main/

■ ■ ■ java/

■ ■ ■ com/eccommerce/

■ ■ ■ base/

■ ■ ■ ■ ■ BaseDriver.java (Singleton WebDriver)

■ ■ ■ ■ ■ BasePage.java (Fluent reusable actions)

■ ■ ■ ■ ■ factory/

■ ■ ■ ■ ■ BrowserFactory.java (Chrome/Firefox/Edge setup)

■ ■ ■ ■ ■ pages/

■ ■ ■ ■ ■ LoginPage.java

■ ■ ■ ■ ■ HomePage.java

■ ■ ■ ■ ■ SearchResultsPage.java

■ ■ ■ ■ ■ ProductPage.java

■ ■ ■ ■ ■ CartPage.java

■ ■ ■ ■ ■ CheckoutPage.java

■ ■ ■ ■ ■ OrderConfirmationPage.java

■ ■ ■ ■ ■ utils/

■ ■ ■ ■ ■ ConfigReader.java (reads config.properties)

- ■■■ ExcelUtils.java (Apache POI data reader)
- ■■■ ScreenshotUtils.java (takes & saves screenshots)
- test/
- java/
- ■■■ com/ecommerce/
- ■■■ listeners/
- ■ ■■■ TestListener.java (ExtentReports hook)
- ■ ■■■ RetryAnalyzer.java (retry on failure)
- ■■■ tests/
- ■■■ LoginTest.java
- ■■■ SearchTest.java
- ■■■ CartTest.java
- ■■■ OrderTest.java
- resources/
- config.properties
- testdata.json
- testdata.xlsx
- log4j2.xml

FILE: src/main/java/com/ecommerce/utils/ConfigReader.java

Page 11

```

import java.io.FileInputStream;

import java.io.IOException;

import java.util.Properties;

public class ConfigReader {

    private static Properties props;

    private static final String CONFIG_PATH =

"src/test/resources/config.properties";

    static {

        try {

            props = new Properties();

            FileInputStream fis = new FileInputStream(CONFIG_PATH);

            props.load(fis);

        } catch (IOException e) {

            throw new RuntimeException("config.properties not found", e);

        }

    }

    public static String get(String key) {

        return props.getProperty(key);

    }

    public static int getInt(String key) {

        return Integer.parseInt(props.getProperty(key));

    }

    public static boolean getBool(String key) {

        return Boolean.parseBoolean(props.getProperty(key));

    }

}

```

FILE: `src/main/java/com/ecommerce/factory/BrowserFactory.java`

```
package com.ecommerce.factory;

import io.github.bonigarcia.wdm.WebDriverManager;

import org.openqa.selenium.WebDriver;

import org.openqa.selenium.chrome.ChromeDriver;

import org.openqa.selenium.chrome.ChromeOptions;

import org.openqa.selenium.firefox.FirefoxDriver;

import org.openqa.selenium.firefox.FirefoxOptions;

import org.openqa.selenium.edge.EdgeDriver;

import org.openqa.selenium.edge.EdgeOptions;

import com.ecommerce.utils.ConfigReader;

public class BrowserFactory {

    public static WebDriver createDriver(String browser) {

        boolean headless = ConfigReader.getBool("headless");

        switch (browser.toLowerCase()) {

            case "chrome":

                WebDriverManager.chromedriver().setup();

                ChromeOptions co = new ChromeOptions();

                co.addArguments("--no-sandbox");

                co.addArguments("--disable-dev-shm-usage");

                co.addArguments("--window-size=1920,1080");

                if (headless) co.addArguments("--headless=new");

                return new ChromeDriver(co);
```

```
case "firefox":

WebDriverManager.firefoxdriver().setup();

FirefoxOptions fo = new FirefoxOptions();

if (headless) fo.addArguments("-headless");

return new FirefoxDriver(fo);


case "edge":

WebDriverManager.edgedriver().setup();

EdgeOptions eo = new EdgeOptions();

if (headless) eo.addArguments("--headless=new");

return new EdgeDriver(eo);


default:

throw new IllegalArgumentException("Unsupported browser: " + browser);

}

}

}
```

BaseDriver – Singleton WebDriver

Thread-safe Singleton holding one driver per thread

FILE: `src/main/java/com/ecommerce/base/BaseDriver.java`

```
package com.ecommerce.base;

import com.ecommerce.factory.BrowserFactory;
import com.ecommerce.utils.ConfigReader;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.support.ui.WebDriverWait;
import java.time.Duration;

public class BaseDriver {

    // ThreadLocal ensures each thread has its own driver (parallel safe)
    private static ThreadLocal<WebDriver> driver = new ThreadLocal<>();

    public static void initDriver() {
        String browser = ConfigReader.get("browser");
        WebDriver d = BrowserFactory.createDriver(browser);
        d.manage().window().maximize();
        d.manage().timeouts().implicitlyWait(
            Duration.ofSeconds(ConfigReader.getInt("implicit.wait")));
        d.manage().timeouts().pageLoadTimeout(
            Duration.ofSeconds(ConfigReader.getInt("page.load.timeout")));
        driver.set(d);
    }

    public static WebDriver getDriver() {
        return driver.get();
    }
}
```

```
public static WebDriverWait getWait() {  
  
    return new WebDriverWait(getDriver(),  
  
        Duration.ofSeconds(ConfigReader.getInt("explicit.wait")));  
  
}  
  
public static void quitDriver() {  
  
    if (driver.get() != null) {  
  
        driver.get().quit();  
  
        driver.remove();  
  
    }  
  
}  
  
}
```

BasePage – Fluent Reusable Actions

All common Selenium actions in one place (Fluent pattern)

FILE: `src/main/java/com/ecommerce/base/BasePage.java`

```
package com.ecommerce.base;

import org.openqa.selenium.*;
import org.openqa.selenium.support.PageFactory;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;
import java.time.Duration;

public abstract class BasePage {

    protected WebDriver driver;

    protected static final Logger log = LogManager.getLogger(BasePage.class);

    public BasePage(WebDriver driver) {

        this.driver = driver;

        // PageFactory initialises all @FindBy elements automatically
        PageFactory.initElements(driver, this);

    }

    // Wait until element is visible then click (Fluent)
    protected BasePage click(WebElement element) {

        BaseDriver.getWait().until(ExpectedConditions.elementToBeClickable(element));

        element.click();

        return this;

    }

    // Clear field then type text
```

```

protected BasePage type(WebElement element, String text) {

    BaseDriver.getWait().until(ExpectedConditions.visibilityOf(element));

    element.clear();

    element.sendKeys(text);

    return this;

}

// Get trimmed text of an element

protected String getText(WebElement element) {

    BaseDriver.getWait().until(ExpectedConditions.visibilityOf(element));

    return element.getText().trim();

}

// Check if element is displayed (no exception thrown)

protected boolean isDisplayed(WebElement element) {

    try {

        return element.isDisplayed();

    } catch (NoSuchElementException e) {

        return false;

    }

}

// Navigate to URL

public void navigateTo(String url) {

    log.info("Navigating to: " + url);

    driver.get(url);

}

// Get current page title

public String getPageTitle() {

    return driver.getTitle();

}

```

```
// Scroll element into view

protected void scrollTo(WebElement element) {

    ((JavascriptExecutor) driver)

        .executeScript("arguments[0].scrollIntoView(true);", element);

}

}
```

One class per page — all locators and actions

8a. LoginPage.java

FILE: src/main/java/com/ecommerce/pages/LoginPage.java

```
package com.ecommerce.pages;

import com.ecommerce.base.BasePage;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.FindBy;

public class LoginPage extends BasePage {

    // ■■ @FindBy locators (PageFactory injects these automatically) ■■

    @FindBy(id = "user-name")
    private WebElement usernameField;

    @FindBy(id = "password")
    private WebElement passwordField;

    @FindBy(id = "login-button")
    private WebElement loginButton;

    @FindBy(css = "[data-test='error']")
    private WebElement errorMessage;

    public LoginPage(WebDriver driver) {
        super(driver);
    }

    // Open login page

    public LoginPage open(String baseUrl) {
```

```

navigateTo(baseUrl);

log.info("Login page opened");

return this;
}

// Enter username

public LoginPage enterUsername(String username) {

type(usernameField, username);

return this;
}

// Enter password

public LoginPage enterPassword(String password) {

type(passwordField, password);

return this;
}

// Click login button – returns HomePage

public HomePage clickLogin() {

click(loginButton);

log.info("Login button clicked");

return new HomePage(driver);
}

// Click login expecting failure (stays on LoginPage)

public LoginPage clickLoginExpectingFailure() {

click(loginButton);

return this;
}

// Get error message text

public String getErrorMessage() {

```

```

return getText(errorMessage);

}

// Is error message visible?

public boolean isErrorDisplayed() {

return isDisplayed(errorMessage);

}

// Full login flow (fluent chaining)

public HomePage loginWith(String username, String password) {

return enterUsername(username).enterPassword(password).clickLogin();

}

}

```

8b. HomePage.java

FILE: src/main/java/com/ecommerce/pages/HomePage.java

```

package com.ecommerce.pages;

import com.ecommerce.base.BasePage;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.FindBy;
import java.util.List;

public class HomePage extends BasePage {

@FindBy(className = "title")

private WebElement pageTitle;

@FindBy(css = ".inventory_item_name")

private List<WebElement> productNames;

@FindBy(css = "button[id^='add-to-cart']")

private List<WebElement> addToCartButtons;

```

```

@FindBy(className = "shopping_cart_link")

private WebElement cartIcon;

@FindBy(css = "[data-test='product_sort_container']")

private WebElement sortDropdown;

public HomePage(WebDriver driver) {

    super(driver);

}

public boolean isPageLoaded() {

    return getText(pageTitle).equals("Products");

}

public int getProductCount() {

    return productNames.size();

}

// Add first product to cart

public HomePage addFirstProductToCart() {

    click(addToCartButtons.get(0));

    return this;

}

// Add product by index

public HomePage addProductToCart(int index) {

    click(addToCartButtons.get(index));

    return this;

}

// Click product name to open product page

public ProductPage clickProduct(int index) {

    click(productNames.get(index));

```

```
return new ProductPage(driver);

}

public CartPage goToCart() {

    click(cartIcon);

    return new CartPage(driver);

}

public String getFirstProductName() {

    return getText(productNames.get(0));

}

}
```

8c. SearchResultsPage.java (Product Filter / Sort)

FILE: src/main/java/com/ecommerce/pages/SearchResultsPage.java

```
package com.ecommerce.pages;

import com.ecommerce.base.BasePage;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.FindBy;
import org.openqa.selenium.support.ui.Select;
import java.util.List;
import java.util.stream.Collectors;

public class SearchResultsPage extends BasePage {

    @FindBy(css = "[data-test='product_sort_container']")
    private WebElement sortDropdown;

    @FindBy(css = ".inventory_item_name")
    private List<WebElement> productNames;

    @FindBy(css = ".inventory_item_price")
    private List<WebElement> productPrices;

    public SearchResultsPage(WebDriver driver) {
        super(driver);
    }

    // Sort products using dropdown

    public SearchResultsPage sortBy(String option) {

        Select select = new Select(sortDropdown);

        select.selectByVisibleText(option);

        log.info("Sorted by: " + option);

        return this;
    }
}
```

```

// Get all product names as list of strings

public List<String> getAllProductNames() {

return productNames.stream()

.map(WebElement::getText)

.collect(Collectors.toList());

}

// Check if product name exists in list

public boolean isProductPresent(String name) {

return getAllProductNames().stream()

.anyMatch(n -> n.equalsIgnoreCase(name));

}

public int getTotalProductCount() {

return productNames.size();

}

}

```

8d. ProductPage.java

FILE: `src/main/java/com/ecommerce/pages/ProductPage.java`

```

package com.ecommerce.pages;

import com.ecommerce.base.BasePage;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.FindBy;

public class ProductPage extends BasePage {

@FindBy(css = ".inventory_details_name")

private WebElement productName;

@FindBy(css = ".inventory_details_price")

```

```

private WebElement productPrice;

@FindBy(css = ".inventory_details_desc")
private WebElement productDescription;

@FindBy(css = "button[id^='add-to-cart']")
private WebElement addToCartButton;

@FindBy(id = "back-to-products")
private WebElement backButton;

public ProductPage(WebDriver driver) {
    super(driver);
}

public String getProductName() {
    return getText(productName);
}

public String getProductPrice() {
    return getText(productPrice);
}

public ProductPage addToCart() {
    click(addToCartButton);

    log.info("Product added to cart: " + getProductName());

    return this;
}

public HomePage goBack() {
    click(backButton);

    return new HomePage(driver);
}
}

```

8e. CartPage.java

FILE: src/main/java/com/ecommerce/pages/CartPage.java

```
package com.ecommerce.pages;

import com.ecommerce.base.BasePage;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.FindBy;
import java.util.List;

public class CartPage extends BasePage {

    @FindBy(css = ".cart_item_label .inventory_item_name")
    private List<WebElement> cartItemNames;

    @FindBy(css = ".inventory_item_price")
    private List<WebElement> cartItemPrices;

    @FindBy(id = "checkout")
    private WebElement checkoutButton;

    @FindBy(id = "continue-shopping")
    private WebElement continueShoppingButton;

    @FindBy(css = ".cart_quantity")
    private List<WebElement> quantities;

    public CartPage(WebDriver driver) {
        super(driver);
    }

    public int getCartItemCount() {
        return cartItemNames.size();
    }
}
```

```

public String getCartItemName(int index) {

    return getText(cartItemNames.get(index));

}

public boolean isItemInCart(String productName) {

    return cartItemNames.stream()

        .anyMatch(e -> e.getText().equals(productName));

}

public CheckoutPage proceedToCheckout() {

    click(checkoutButton);

    log.info("Proceeding to checkout");

    return new CheckoutPage(driver);

}

public HomePage continueShopping() {

    click(continueShoppingButton);

    return new HomePage(driver);

}

}

```

8f. CheckoutPage.java

FILE: `src/main/java/com/ecommerce/pages/CheckoutPage.java`

```

package com.ecommerce.pages;

import com.ecommerce.base.BasePage;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.FindBy;

public class CheckoutPage extends BasePage {

    @FindBy(id = "first-name")

    private WebElement firstNameField;

```

```

@FindBy(id = "last-name")

private WebElement lastNameField;

@FindBy(id = "postal-code")

private WebElement postalCodeField;

@FindBy(id = "continue")

private WebElement continueButton;

@FindBy(id = "finish")

private WebElement finishButton;

@FindBy(css = ".summary_info")

private WebElement orderSummary;

@FindBy(css = ".summary_total_label")

private WebElement totalLabel;

public CheckoutPage(WebDriver driver) {

    super(driver);

}

public CheckoutPage enterShippingInfo(

    String firstName, String lastName, String postalCode) {

    type(firstNameField, firstName);

    type(lastNameField, lastName);

    type(postalCodeField, postalCode);

    return this;

}

public CheckoutPage clickContinue() {

    click(continueButton);

    return this;

```

```
}

public String getTotal() {
    return getText(totalLabel);
}

public OrderConfirmationPage clickFinish() {
    click(finishButton);

    log.info("Order placed - finish clicked");
    return new OrderConfirmationPage(driver);
}
}
```

8g. OrderConfirmationPage.java

FILE: src/main/java/com/ecommerce/pages/OrderConfirmationPage.java

```
package com.ecommerce.pages;

import com.ecommerce.base.BasePage;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.FindBy;

public class OrderConfirmationPage extends BasePage {

    @FindBy(css = ".complete-header")
    private WebElement confirmationHeader;

    @FindBy(css = ".complete-text")
    private WebElement confirmationText;

    @FindBy(id = "back-to-products")
    private WebElement backButton;

    public OrderConfirmationPage(WebDriver driver) {
        super(driver);
    }

    public String getConfirmationHeader() {
        return getText(confirmationHeader);
    }

    public String getConfirmationMessage() {
        return getText(confirmationText);
    }

    public boolean isOrderConfirmed() {
        String header = getConfirmationHeader();

        log.info("Order confirmation header: " + header);
    }
}
```

```
return header.contains("Thank you") || header.contains("THANK YOU");  
  
}  
  
public HomePage backToProducts() {  
  
    click(backButton);  
  
    return new HomePage(driver);  
  
}  
  
}
```

testdata.json

FILE: src/test/resources/testdata.json

```
{
  "loginData": [
    {
      "testCase": "Valid Login",
      "username": "standard_user",
      "password": "secret_sauce",
      "expectedResult": "success"
    },
    {
      "testCase": "Invalid Username",
      "username": "wrong_user",
      "password": "secret_sauce",
      "expectedResult": "failure"
    },
    {
      "testCase": "Invalid Password",
      "username": "standard_user",
      "password": "wrong_pass",
      "expectedResult": "failure"
    },
    {
      "testCase": "Locked Out User",
      "username": "locked_out_user",
      "password": "secret_sauce",
```

```

    "expectedResult": "failure"

  },

  {

    "testCase": "Empty Credentials",

    "username": "",

    "password": "",

    "expectedResult": "failure"

  }

],

"searchData": [

  { "product": "Sauce Labs Backpack" },

  { "product": "Sauce Labs Bike Light" },

  { "product": "Sauce Labs Bolt T-Shirt" }

],

"shippingData": {

  "firstName": "John",

  "lastName": "Doe",

  "postalCode": "110001"

}

}

```

ExcelUtils.java (Apache POI)

FILE: `src/main/java/com/ecommerce/utils/ExcelUtils.java`

```

package com.ecommerce.utils;

import org.apache.poi.ss.usermodel.*;
import org.apache.poi.xssf.usermodel.XSSFWorkbook;

import java.io.*;
import java.util.*;

public class ExcelUtils {

```

```

private static Workbook workbook;

private static Sheet sheet;

public static void setExcelFile(String path, String sheetName)
throws IOException {

    FileInputStream fis = new FileInputStream(path);

    workbook = new XSSFWorkbook(fis);

    sheet = workbook.getSheet(sheetName);

}

// Returns 2D Object array for TestNG DataProvider

public static Object[][] getTableData(String path, String sheetName)
throws IOException {

    setExcelFile(path, sheetName);

    int rowCount = sheet.getLastRowNum();

    int colCount = sheet.getRow(0).getLastCellNum();

    Object[][] data = new Object[rowCount][colCount];

    for (int i = 1; i <= rowCount; i++) {

        Row row = sheet.getRow(i);

        for (int j = 0; j < colCount; j++) {

            Cell cell = row.getCell(j);

            data[i-1][j] = (cell != null)

            ? cell.toString() : "";

        }

    }

    return data;

}

public static String getCellData(int row, int col) {

    Cell cell = sheet.getRow(row).getCell(col);

    return (cell != null) ? cell.toString() : "";

```

```
}
```

```
}
```

10a. LoginTest.java

FILE: `src/test/java/com/ecommerce/tests/LoginTest.java`

```
package com.ecommerce.tests;

import com.ecommerce.base.BaseDriver;

import com.ecommerce.listeners.RetryAnalyzer;

import com.ecommerce.pages.HomePage;

import com.ecommerce.pages.LoginPage;

import com.ecommerce.utils.ConfigReader;

import org.testng.Assert;

import org.testng.annotations.*;

import org.apache.logging.log4j.LogManager;

import org.apache.logging.log4j.Logger;

@Listeners(com.ecommerce.listeners.TestListener.class)

public class LoginTest {

    private static final Logger log = LogManager.getLogger(LoginTest.class);

    private LoginPage loginPage;

    @BeforeMethod

    public void setUp() {

        BaseDriver.initDriver();

        loginPage = new LoginPage(BaseDriver.getDriver());

        loginPage.open(ConfigReader.get("base.url"));

    }

    @AfterMethod
```

```

public void tearDown() {

    BaseDriver.quitDriver();

}

// DataProvider: valid login datasets

@DataProvider(name = "validLoginData")

public Object[][] validLoginData() {

    return new Object[][] {

        { "standard_user", "secret_sauce" },

        { "performance_glitch_user", "secret_sauce" },

    };

}

// DataProvider: invalid login datasets

@DataProvider(name = "invalidLoginData")

public Object[][] invalidLoginData() {

    return new Object[][] {

        { "wrong_user", "secret_sauce", "Username and password" },

        { "standard_user", "wrong_pass", "Username and password" },

        { "locked_out_user", "secret_sauce", "locked out" },

        { "", "", "Username is required" },

        { "standard_user", "", "Password is required" },

    };

}

@Test(dataProvider = "validLoginData",

    description = "TC_LOGIN_001: Valid credentials should navigate to Products page",

    retryAnalyzer = RetryAnalyzer.class)

public void testValidLogin(String username, String password) {

    log.info("Testing valid login: " + username);

    HomePage homePage = loginPage.loginWith(username, password);

```

```

Assert.assertTrue(homePage.isPageLoaded(),

"Products page should be loaded after valid login");

}

@Test(dataProvider = "invalidLoginData",

description = "TC_LOGIN_002: Invalid credentials should show error message",

retryAnalyzer = RetryAnalyzer.class)

public void testInvalidLogin(String username, String password, String expectedErr) {

log.info("Testing invalid login: " + username);

loginPage.enterUsername(username)

.enterPassword(password)

.clickLoginExpectingFailure();

Assert.assertTrue(loginPage.isErrorDisplayed(),

"Error message should be shown for invalid credentials");

Assert.assertTrue(

loginPage.getErrorMessage().contains(expectedErr),

"Error should contain: " + expectedErr);

}

}

```

10b. SearchTest.java

FILE: src/test/java/com/ecommerce/tests/SearchTest.java

```
package com.ecommerce.tests;

import com.ecommerce.base.BaseDriver;
import com.ecommerce.listeners.RetryAnalyzer;
import com.ecommerce.pages.*;
import com.ecommerce.utils.ConfigReader;
import org.testng.Assert;
import org.testng.annotations.*;
import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;

@Listeners(com.ecommerce.listeners.TestListener.class)
public class SearchTest {

    private static final Logger log = LogManager.getLogger(SearchTest.class);

    private HomePage homePage;

    @BeforeMethod

    public void setUp() {

        BaseDriver.initDriver();

        LoginPage lp = new LoginPage(BaseDriver.getDriver());

        lp.open(ConfigReader.get("base.url"));

        homePage = lp.loginWith(
            ConfigReader.get("valid.username"),
            ConfigReader.get("valid.password"));
    }

    @AfterMethod

    public void tearDown() {

        BaseDriver.quitDriver();
    }
}
```

```

@DataProvider(name = "sortOptions")

public Object[][] sortOptions() {

return new Object[][] {

{ "Name (A to Z)" },

{ "Name (Z to A)" },

{ "Price (low to high)" },

{ "Price (high to low)" },

};

}

@Test(description = "TC_SEARCH_001: Products page should load with items")

public void testProductsPageLoads() {

Assert.assertTrue(homePage.isPageLoaded(),

"Home/Products page should be loaded");

Assert.assertTrue(homePage.getProductCount() > 0,

"At least one product should be visible");

log.info("Products found: " + homePage.getProductCount());

}

@Test(dataProvider = "sortOptions",

description = "TC_SEARCH_002: Products should be sortable by multiple options")

public void testSortProducts(String sortOption) {

SearchResultsPage srp = new SearchResultsPage(BaseDriver.getDriver());

srp.sortBy(sortOption);

Assert.assertTrue(srp.getTotalProductCount() > 0,

"Products should still be visible after sort: " + sortOption);

log.info("Sort by [" + sortOption + "] passed. Count: " + srp.getTotalProductCount());

}

@Test(description = "TC_SEARCH_003: Product detail page should open correctly")

public void testProductDetailPage() {

```

```
String firstName = homePage.getFirstProductName();

ProductPage pp = homePage.clickProduct(0);

Assert.assertEquals(pp.getProductName(), firstName,
    "Product detail name should match listing name");
}

}
```

10c. CartTest.java

FILE: src/test/java/com/ecommerce/tests/CartTest.java

```
package com.ecommerce.tests;

import com.ecommerce.base.BaseDriver;
import com.ecommerce.pages.*;
import com.ecommerce.utils.ConfigReader;
import org.testng.Assert;
import org.testng.annotations.*;
import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;

@Listeners(com.ecommerce.listeners.TestListener.class)
public class CartTest {

    private static final Logger log = LogManager.getLogger(CartTest.class);

    private HomePage homePage;

    private String firstProductName;

    @BeforeMethod

    public void setUp() {

        BaseDriver.initDriver();

        LoginPage lp = new LoginPage(BaseDriver.getDriver());

        lp.open(ConfigReader.get("base.url"));

        homePage = lp.loginWith(

            ConfigReader.get("valid.username"),

            ConfigReader.get("valid.password"));

        firstProductName = homePage.getFirstProductName();

    }

    @AfterMethod

    public void tearDown() {

        BaseDriver.quitDriver();

    }

}
```

```

}

@Test(description = "TC_CART_001: Adding a product should appear in cart")

public void testAddProductToCart() {

    homePage.addFirstProductToCart();

    CartPage cartPage = homePage.goToCart();

    Assert.assertEquals(cartPage.getCartItemCount(), 1,

        "Cart should have exactly 1 item");

    Assert.assertTrue(cartPage.isItemInCart(firstProductName),

        "First product should be in cart");

}

@Test(description = "TC_CART_002: Multiple products can be added to cart")

public void testAddMultipleProducts() {

    homePage.addProductToCart(0);

    homePage.addProductToCart(1);

    CartPage cartPage = homePage.goToCart();

    Assert.assertEquals(cartPage.getCartItemCount(), 2,

        "Cart should have 2 items");

}

@Test(description = "TC_CART_003: Continue shopping returns to products page")

public void testContinueShopping() {

    homePage.addFirstProductToCart();

    CartPage cartPage = homePage.goToCart();

    HomePage hp = cartPage.continueShopping();

    Assert.assertTrue(hp.isPageLoaded(),

        "Should return to Products page");

}

}

```

10d. OrderTest.java

FILE: src/test/java/com/ecommerce/tests/OrderTest.java

```
package com.ecommerce.tests;

import com.ecommerce.base.BaseDriver;
import com.ecommerce.pages.*;
import com.ecommerce.utils.ConfigReader;
import org.testng.Assert;
import org.testng.annotations.*;
import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;

@Listeners(com.ecommerce.listeners.TestListener.class)
public class OrderTest {

    private static final Logger log = LogManager.getLogger(OrderTest.class);

    private HomePage homePage;

    @BeforeMethod
    public void setUp() {

        BaseDriver.initDriver();

        LoginPage lp = new LoginPage(BaseDriver.getDriver());

        lp.open(ConfigReader.get("base.url"));

        homePage = lp.loginWith(
            ConfigReader.get("valid.username"),
            ConfigReader.get("valid.password"));

        homePage.addFirstProductToCart();

    }

    @AfterMethod
    public void tearDown() {

        BaseDriver.quitDriver();

    }
}
```

```

@Test(description = "TC_ORDER_001: Full end-to-end order flow",
priority = 1)

public void testCompleteOrderFlow() {

    log.info("Starting E2E order flow test");

    // Step 1: Go to cart

    CartPage cartPage = homePage.goToCart();

    Assert.assertTrue(cartPage.getCartItemCount() > 0,
        "Cart should not be empty");

    // Step 2: Proceed to checkout

    CheckoutPage checkoutPage = cartPage.proceedToCheckout();

    // Step 3: Fill shipping info

    checkoutPage.enterShippingInfo("John", "Doe", "110001")
        .clickContinue();

    // Step 4: Verify total exists

    String total = checkoutPage.getTotal();

    Assert.assertNotNull(total, "Total amount should be displayed");

    log.info("Order total: " + total);

    // Step 5: Finish order

    OrderConfirmationPage confirmPage = checkoutPage.clickFinish();

    // Step 6: Verify confirmation

    Assert.assertTrue(confirmPage.isOrderConfirmed(),
        "Order confirmation page should show success message");

    log.info("Order confirmed: " + confirmPage.getConfirmationHeader());

}

@Test(description = "TC_ORDER_002: Order confirmation message text validation",
priority = 2)

```

```
public void testOrderConfirmationMessage() {

    CartPage cartPage = homePage.goToCart();

    CheckoutPage checkoutPage = cartPage.proceedToCheckout();

    checkoutPage.enterShippingInfo("Jane", "Smith", "400001")

        .clickContinue();

    OrderConfirmationPage confirmPage = checkoutPage.clickFinish();

    String message = confirmPage.getConfirmationMessage();

    Assert.assertFalse(message.isEmpty(),

        "Confirmation message should not be empty");

}

}
```

ScreenshotUtils.java

FILE: src/main/java/com/ecommerce/utils/ScreenshotUtils.java

```
package com.ecommerce.utils;

import com.ecommerce.base.BaseDriver;

import org.openqa.selenium.OutputType;

import org.openqa.selenium.TakesScreenshot;

import org.apache.commons.io.FileUtils;

import java.io.File;

import java.io.IOException;

import java.time.LocalDateTime;

import java.time.format.DateTimeFormatter;

public class ScreenshotUtils {

    public static String captureScreenshot(String testName) {

        String timestamp = LocalDateTime.now()

            .format(DateTimeFormatter.ofPattern("yyyyMMdd_HH:mm:ss"));

        String path = ConfigReader.get("screenshots.dir")

            + "/" + testName + "_" + timestamp + ".png";

        try {

            File src = ((TakesScreenshot) BaseDriver.getDriver())

                .getScreenshotAs(OutputType.FILE);

            FileUtils.copyFile(src, new File(path));

        } catch (IOException e) {

            System.err.println("Screenshot failed: " + e.getMessage());
        }
    }
}
```

```

    }

    return path;

}

}

```

TestListener.java (ExtentReports + Screenshots)

FILE: `src/test/java/com/ecommerce/listeners/TestListener.java`

```

package com.ecommerce.listeners;

import com.aventstack.extentreports.*;
import com.aventstack.extentreports.reporter.ExtentSparkReporter;
import com.aventstack.extentreports.reporter.configuration.Theme;
import com.ecommerce.utils.ScreenshotUtils;
import org.testng.*;
import java.io.File;

public class TestListener implements ITestListener, ISuiteListener {

    private static ExtentReports extent;

    private static ThreadLocal<ExtentTest> test = new ThreadLocal<>();

    @Override

    public void onStart(ISuite suite) {

        new File("reports").mkdirs();

        new File("screenshots").mkdirs();

        ExtentSparkReporter reporter =

        new ExtentSparkReporter("reports/ExtentReport.html");

        reporter.config().setDocumentTitle("E2E Automation Report");

        reporter.config().setReportName("Ecommerce Test Results");

        reporter.config().setTheme(Theme.DARK);

        extent = new ExtentReports();

        extent.attachReporter(reporter);

```

```

extent.setSystemInfo("OS", System.getProperty("os.name"));

extent.setSystemInfo("Java", System.getProperty("java.version"));

extent.setSystemInfo("Env", "QA");

}

@Override

public void onFinish(ISuite suite) {

    extent.flush();

}

@Override

public void onStart(ITestResult result) {

    ExtentTest t = extent.createTest(result.getMethod().getMethodName(),

    result.getMethod().getDescription());

    test.set(t);

}

@Override

public void onTestSuccess(ITestResult result) {

    test.get().pass("Test PASSED");

}

@Override

public void onTestFailure(ITestResult result) {

    // Capture screenshot on failure

    String screenshotPath = ScreenshotUtils.captureScreenshot(

    result.getMethod().getMethodName());

    test.get().fail(result.getThrowable())

    .addScreenCaptureFromPath(screenshotPath, "Failure Screenshot");

}

@Override

```

```
public void onTestSkipped(ITestResult result) {  
  
    test.get().skip(result.getThrowable());  
  
}  
  
}
```

RetryAnalyzer – Retry on Failure

Bonus: automatic retry mechanism for flaky tests

FILE: `src/test/java/com/ecommerce/listeners/RetryAnalyzer.java`

```
package com.ecommerce.listeners;

import org.testng.IRetryAnalyzer;
import org.testng.ITestResult;

public class RetryAnalyzer implements IRetryAnalyzer {

    private int retryCount = 0;

    private static final int MAX_RETRY = 2; // retries up to 2 times

    @Override

    public boolean retry(ITestResult result) {

        if (retryCount < MAX_RETRY) {

            retryCount++;

            System.out.println("Retrying test ["

            + result.getName()

            + "] attempt " + retryCount);

            return true; // retry the test

        }

        return false; // do not retry

    }

}
```

How to Apply RetryAnalyzer

Add `retryAnalyzer = RetryAnalyzer.class` to any `@Test` annotation:

```
// On individual test:

@Test(retryAnalyzer = RetryAnalyzer.class)
```

```
public void myTest() { ... }
```

```
// Already added in LoginTest.java example above.
```

```
// For global retry, configure in testng.xml listener section.
```

testng.xml – Suite Configuration

Parallel execution, groups, listeners

FILE: testng.xml (root of project, same level as pom.xml)

```
<?xml version="1.0" encoding="UTF-8"?>

<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">

<suite name="EcommerceTestSuite" verbose="1"
parallel="methods" thread-count="3">

  <!-- Attach listeners globally -->

  <listeners>

    <listener class-name="com.ecommerce.listeners.TestListener"/>

  </listeners>

  <!-- Group 1: Smoke Tests -->

  <test name="SmokeTests">

    <groups>

      <run><include name="smoke"/></run>

    </groups>

    <classes>

      <class name="com.ecommerce.tests.LoginTest"/>

    </classes>

  </test>

  <!-- Group 2: Regression Tests -->

  <test name="RegressionTests">

    <classes>

      <class name="com.ecommerce.tests.LoginTest"/>

      <class name="com.ecommerce.tests.SearchTest"/>

      <class name="com.ecommerce.tests.CartTest"/>

    </classes>

  </test>

</suite>
```

```
<class name="com.ecommerce.tests.OrderTest"/>

</classes>

</test>

<!-- Cross-Browser: Run on Firefox -->

<test name="FirefoxTests">

  <parameter name="browser" value="firefox"/>

  <classes>

    <class name="com.ecommerce.tests.OrderTest"/>

  </classes>

</test>

</suite>
```

NOTE: parallel='methods' with thread-count='3' runs up to 3 test methods simultaneously. ThreadLocal WebDriver in BaseDriver ensures each thread has its own browser instance.

log4j2.xml – Logging Configuration

Console + rolling file logging with SLF4J

FILE: `src/test/resources/log4j2.xml`

```
<?xml version="1.0" encoding="UTF-8"?>

<Configuration status="WARN">

  <Appenders>

    <!-- Console output with colours -->

    <Console name="Console" target="SYSTEM_OUT">

      <PatternLayout

        pattern="%d{HH:mm:ss} [%t] %-5level %logger{36} - %msg%n"/>

      </Console>

    <!-- Rolling file – new log each day, keeps 7 days -->

    <RollingFile name="FileLog"

      fileName="logs/automation.log"

      filePattern="logs/automation-%d{yyyy-MM-dd}.log.gz">

      <PatternLayout>

        <Pattern>%d{yyyy-MM-dd HH:mm:ss} [%t] %-5level %logger - %msg%n</Pattern>

      </PatternLayout>

      <Policies>

        <TimeBasedTriggeringPolicy />

      </Policies>

      <DefaultRolloverStrategy max="7"/>

    </RollingFile>

  </Appenders>

  <Loggers>
```

```
<Root level="INFO">

<AppenderRef ref="Console"/>

<AppenderRef ref="FileLog"/>

</Root>

<!-- Silence noisy Selenium logs -->

<Logger name="org.openqa.selenium" level="WARN" additivity="false">

<AppenderRef ref="Console"/>

</Logger>

</Loggers>

</Configuration>
```

Jenkinsfile – CI/CD Pipeline

Declarative pipeline with stages, reporting, and artifact archiving

FILE: Jenkinsfile (root of project)

```
pipeline {  
  
    agent any  
  
    environment {  
  
        MAVEN_OPTS = '-Xmx1024m'  
  
    }  
  
    tools {  
  
        maven 'Maven-3.9'  
  
        jdk 'JDK-11'  
  
    }  
  
    stages {  
  
        stage('Checkout') {  
  
            steps {  
  
                git branch: 'main',  
  
                url: 'https://github.com/your-repo/EcommerceAutomation.git'  
  
            }  
  
        }  
  
        stage('Build') {  
  
            steps {  
  
                sh 'mvn clean compile -q'  
  
            }  
  
        }  
  
        stage('Run Tests') {
```

```

steps {

sh 'mvn test -Dheadless=true'

}

post {

always {

// Publish TestNG HTML report

publishHTML(target: [

reportDir: 'reports',

reportFiles: 'ExtentReport.html',

reportName: 'ExtentReport',

keepAll: true

])

}

}

}

stage('Archive Artifacts') {

steps {

archiveArtifacts artifacts: 'reports/**,screenshots/**,logs/**',

allowEmptyArchive: true

}

}

post {

failure {

echo 'Tests FAILED. Check ExtentReport for details.'

}

success {

echo 'All tests PASSED!'

}

}

```

```
}
```

```
}
```

```
}
```

GitHub Actions – CI Workflow

.github/workflows/ci.yml

FILE: .github/workflows/ci.yml

```
name: E2E Automation CI

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up JDK 11
        uses: actions/setup-java@v4
        with:
          java-version: '11'
          distribution: 'temurin'

      - name: Cache Maven packages
        uses: actions/cache@v4
        with:
          path: ~/.m2
          key: ${{ runner.os }}-maven-${{ hashFiles('**/pom.xml') }}
```

```
- name: Install Chrome

uses: browser-actions/setup-chrome@v1


- name: Run Tests (headless Chrome)

run: mvn test -Dheadless=true -Dbrowser=chrome


- name: Upload Extent Report

uses: actions/upload-artifact@v4

if: always()

with:

name: ExtentReport

path: reports/


- name: Upload Screenshots

uses: actions/upload-artifact@v4

if: failure()

with:

name: FailureScreenshots

path: screenshots/
```

Run Instructions & Troubleshooting

How to execute the framework end-to-end

Running Tests in Eclipse

Step 1: Right-click testng.xml → Run As → TestNG Suite

Step 2: Or right-click any Test class → Run As → TestNG Test

Step 3: Reports generated at: reports/ExtentReport.html

Step 4: Logs generated at: logs/automation.log

Step 5: Screenshots saved at: screenshots/ (only on failure)

Running via Maven Command Line

```
# Run all tests

mvn clean test

# Run with headless browser (for VM/CI)

mvn clean test -Dheadless=true

# Run specific browser

mvn clean test -Dbrowser=firefox

# Run specific test class

mvn -Dtest=LoginTest test

# Run specific method

mvn -Dtest=LoginTest#testValidLogin test
```

Troubleshooting Common Issues

ChromeDriver version mismatch	WebDriverManager auto-handles this. Ensure internet access in VM.
NoSuchElementException	Increase implicit.wait in config.properties. Check locator with browser DevTools.

StaleElementReferenceException	Re-initialise page object in the test. Avoid storing WebElements across page navigations.
Tests running in wrong order	Use priority=1,2,3 in @Test or dependsOnMethods='method'
ExtentReport not generated	Ensure TestListener is declared in testng.xml block.
Maven dependencies not downloading	Right-click → Maven → Update Project. Check internet in VM. Try: mvn dependency:resolve
Parallel tests fail	Check ThreadLocal is used in BaseDriver. Never use static WebDriver.

Deliverables Checklist

- Complete framework (all Java source files as described)
- testng.xml (suite with parallel execution configured)
- reports/ExtentReport.html (generated after test run)
- logs/automation.log (per-run log file)
- screenshots/ (PNG screenshots captured on failure)
- Jenkinsfile (declarative pipeline for Jenkins CI)
- .github/workflows/ci.yml (GitHub Actions pipeline)
- Bonus: RetryAnalyzer.java (automatic retry on failure)
- Bonus: Cross-browser support Chrome / Firefox / Edge
- Bonus: Environment config QA/UAT/Prod via config.properties

End of Guide — E2E Automation Framework Complete

Java | Selenium | TestNG | Maven | ExtentReports | Log4j2 | CI/CD